CI/CD IN MLOPS

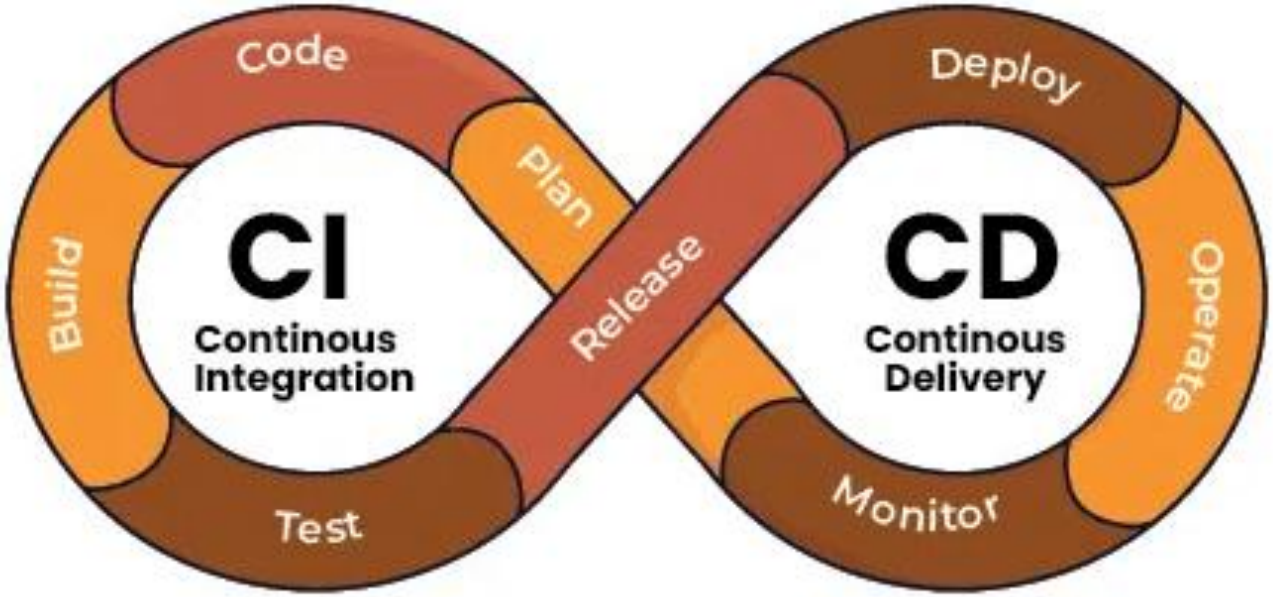
© 2011-25 National University of Singapore. All Rights Reserved.

Topics

- Introduction to MLOps
- **Pipeline design**
- Monitoring and Logging
- MLOps with Cloud platforms

© 2011-24 National University of Singapore. All Rights Reserved.

Recap CI/CD in Software Engineering



Source: https://www.geeksforgeeks.org/system-design/cicd-pipeline-system-design/

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Continuous Integration

- Integration of Code
- Automated Testing
- Feedback

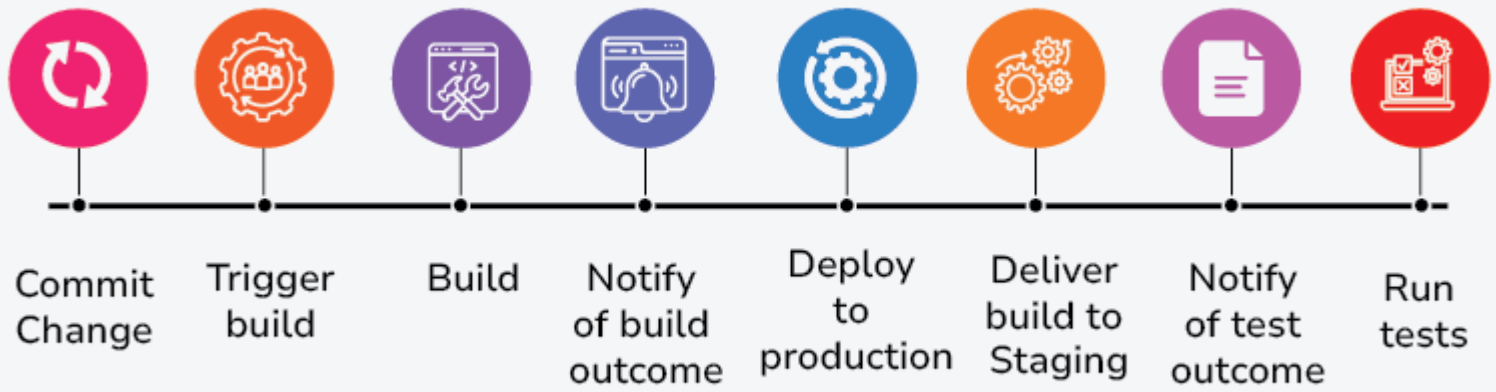
Continuous Deployment

- Continuous Delivery
- Continuous Deployment

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Pipeline Flow – for software engineering



Source: https://www.geeksforgeeks.org/system-design/cicd-pipeline-system-design/

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Commit Change

- Push code changes to version control like Git

Trigger Build

- Automated triggering the build process

Build

- Compile
- Build into deployable artifacts such as docker image

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Notification of Build Outcome

- Sending build result via email, Slack etc

Run Tests

- Automated tests such as Unit Tests, Integration Tests, End-to-End tests

Notification of Test outcome

- Feedback on test outcomes allows rectification before deployment
- Results can be sent via email, chat, or dashboards

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Deliver Build to Staging

- Deployment to staging environment

Deploy to Production

- Promoted to production after successful validation in staging environment

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Manual MLOps process – Experimental Stage

Data Store

Data for analysis and model development is stored,
such as large volumes of data in Hadoop, from which data scientists can pull raw data for experiments

Raw Data Processing

Raw data needs to be processed to extract relevant information, purge faults,
and remove corrupted entries, as these can harm data analysis and model development

Data Analysis

Execute training whilst periodically evaluating against a validation set.

Model Development

Building, training, evaluating, and validating the model. Once finalized, a workable model class is created
and pushed to a code repository.

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Manual MLOps process – Deployment Stage

Model Deployment

manually integrate the model into the existing application or system

Model Services

Deployed model interacts with user in real time and operation team monitors the model's performance

Data Collection

Operation Team collects raw data raw data and performance metrics

which are vital for business decisions, such as service popularity or model performance

Data forwarded to Data Store

Collected data is sent to the data store

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Pain points of MLOps process

- **Work-Intensive and Resource-Heavy**
- **Frequent Retraining Needs**
- **Difficulty with Changing Data Trends**
- **Difficult to keep track of performance and organize results**
- **Complex Model Integration**
- **Lack of Continuous Delivery**

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

MLOps Pipeline

- Continuous Integration (CI) in MLOps
- Continuous Deployment (CD) in MLOps

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

CI in MLOps

Build the model artifacts

Bundling the model code, preprocessing steps, hyperparameters, training/validation data, the trained model, its environment (libraries, versions), and documentation

Send the artifacts to permanent storage

For future access and traceability

Run basic checks

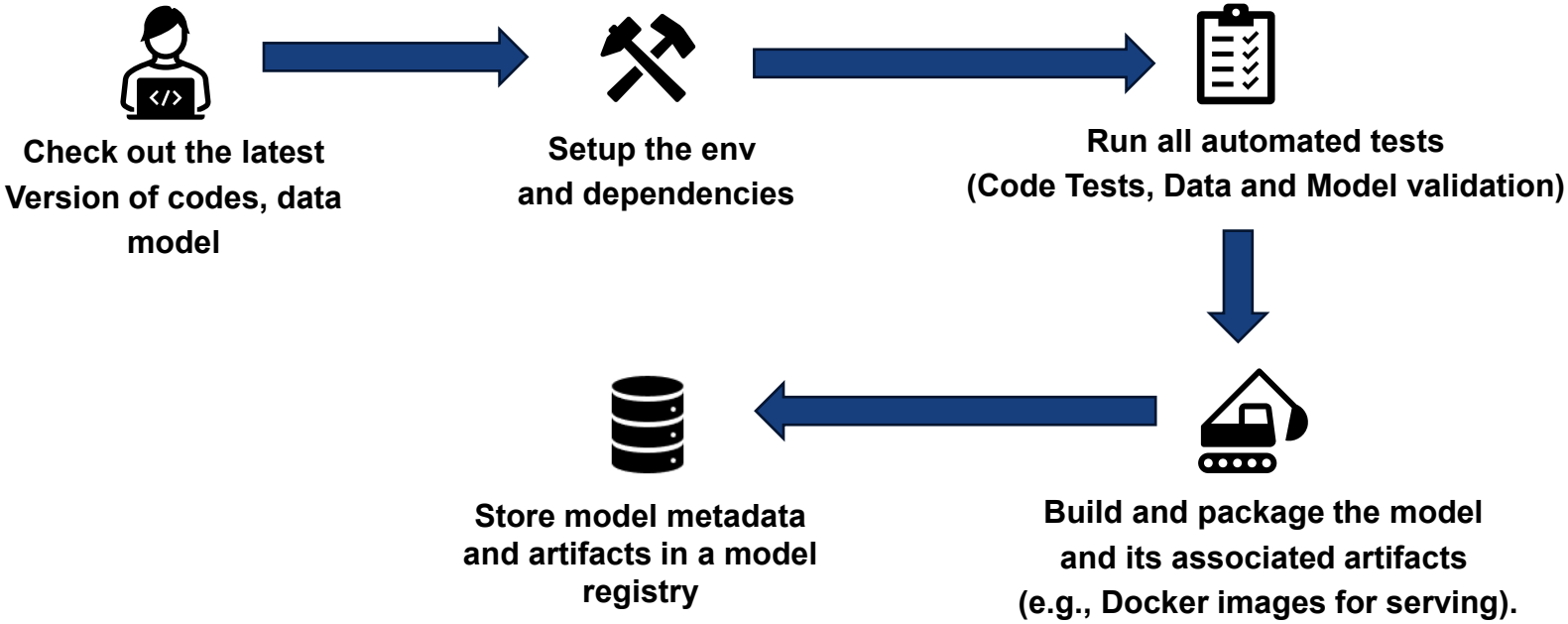
Execute training whilst periodically evaluating against a validation set.

Model Development

Building, training, evaluating, and validating the model. Once finalized, a workable model class is created and pushed to a code repository.

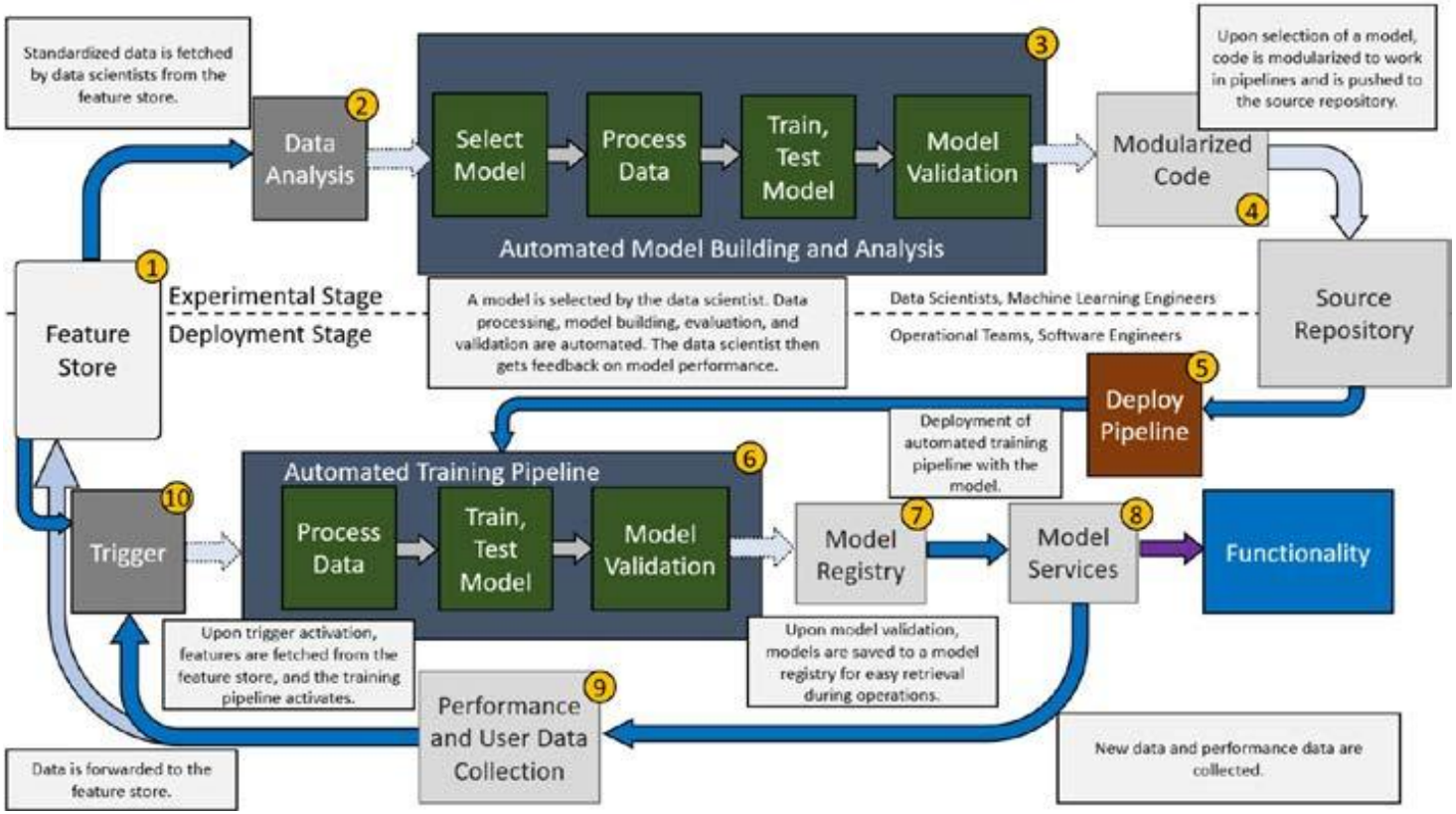
© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps
CD in MLOps



© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps
MLOps CI/CD Pipeline



Source: Beginning MLOps with MLFlow

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Feature Store

- Centralized repository that manages, stores, and serves features for ML models
- Ensure that features used for model training are consistent and available

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Model Registry

- Storage unit that specifically holds model classes and/or weights.

- Purpose of this unit is to hold trained models for easy retrieval by an application

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Model Services

- Here the application pulls the latest, best performing model from the model registry and makes use of its prediction services

- This action then goes on to provide the desired functionality in the application

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD In MLOps

Assignment 1

- Design an end-to-end CI/CD Pipeline for a ML model
- Describe clearly what needs to be done in each task.
- Refer to Assignment instructions for more details

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD in MLOps

Implement CI/CD in MLOps

Stage	Tool Category	Tools	Purpose
1. Version Control	VCS	Git, GitHub, GitLab, Bitbucket	Track code, models, configs
2. CI/CD Orchestration	CI/CD Engines	GitHub Actions, GitLab CI/CD, Jenkins, CircleCI	Automate testing, linting, packaging
3. Data & Feature Management	Data/Feature Store	Feast	Version and manage datasets/features
4. Experiment Tracking	Tracking & Metadata	MLflow	Log metrics, params, artifacts
5. Model Registry	Model Registry	MLflow	Store, version, and stage models
6. Pipeline Orchestration	Workflow/Pipeline Engine	Kubeflow Pipelines	Define and manage ML workflows
7. Testing	ML Testing Frameworks	Pytest, JUnit	Data and model testing
8. Containerization	Environment Packaging	Docker	Package and isolate environments
9. Deployment	Model Serving	FastAPI, Flask	Deploy models as APIs
10. Monitoring	Model/Data Monitoring	Evidently AI (Drift Monitoring) , Prometheus + Grafana (System monitoring)	Monitor drift, performance, alerts

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD in MLOps

Implement CI/CD Using MLFlow

MLFlow is an open-source python library that helps to manage ML lifecycle :

- Experiment Tracking
- Model Packaging
- Model Registry
- Deployment

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD in MLOps

Implement CI/CD Using MLFlow

Installing MLFlow

pip install mlflow

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow

A simple loan application dataset. The goal given to a ML engineer is to develop a ML model to predict the approval outcome based on the features shown below.

ID	Age	Has_job	Own_house	Credit_rating	Outcome
1	young	false	false	fair	Reject
2	young	false	false	good	Reject
3	young	true	false	good	Approve
4	young	true	true	fair	Approve
5	young	false	false	fair	Reject
6	middle	false	false	fair	Reject
7	middle	false	false	good	Reject
8	middle	true	true	good	Approve
9	middle	false	true	excellent	Approve
10	middle	false	true	excellent	Approve
11	old	false	true	excellent	Approve
12	old	false	true	good	Approve
13	old	true	false	good	Approve
14	old	true	false	excellent	Approve
15	old	false	false	fair	Reject

features

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Import the libraries

The ML engineer decide to select Decision Tree classifier as the ML algorithm.

Let's see how the ML Engineer integrates his python code with MLFlow.

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import mlflow
```

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow

```
# Loading and Data pre-processing
# Load the dataset
df = pd.read_csv('bank_loan_appln.csv’)
from sklearn.calibration import LabelEncoder

#Drop those columns that are deemed to be useless to our analysis
X = df.drop(columns=['Outcome','ID'], axis=1)
y = df['Outcome']

# Perform encoding
le = LabelEncoder()

# Encode categorical features
columns_to_convert = ['Age','Has_job','Own_house','Credit_rating']
for column in columns_to_convert:
X[column] = le.fit_transform(df[column])
```

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow

```
# Loading and Data pre-processing
# Load the dataset
df = pd.read_csv('bank_loan_appln.csv’)
from sklearn.calibration import LabelEncoder

#Drop those columns that are deemed to be useless to our analysis
X = df.drop(columns=['Outcome','ID'], axis=1)
y = df['Outcome']

# Perform encoding
le = LabelEncoder()

# Encode categorical features
columns_to_convert = ['Age','Has_job','Own_house','Credit_rating']
for column in columns_to_convert:
X[column] = le.fit_transform(df[column])
```

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Start and name the experiment

```
# set the experiment name
mlflow.set_experiment("Bank_Loan_Application_Prediction")

# start the run and name it as run
with mlflow.start_run() as run:

    #hyperparameters can be tuned as needed
    maxDepth = 5

    # Train the Decision Tree Classifier
    dt_classifier = DecisionTreeClassifier(max_depth=maxDepth,
    random_state=42)
    dt_classifier.fit(X_train, y_train)
```

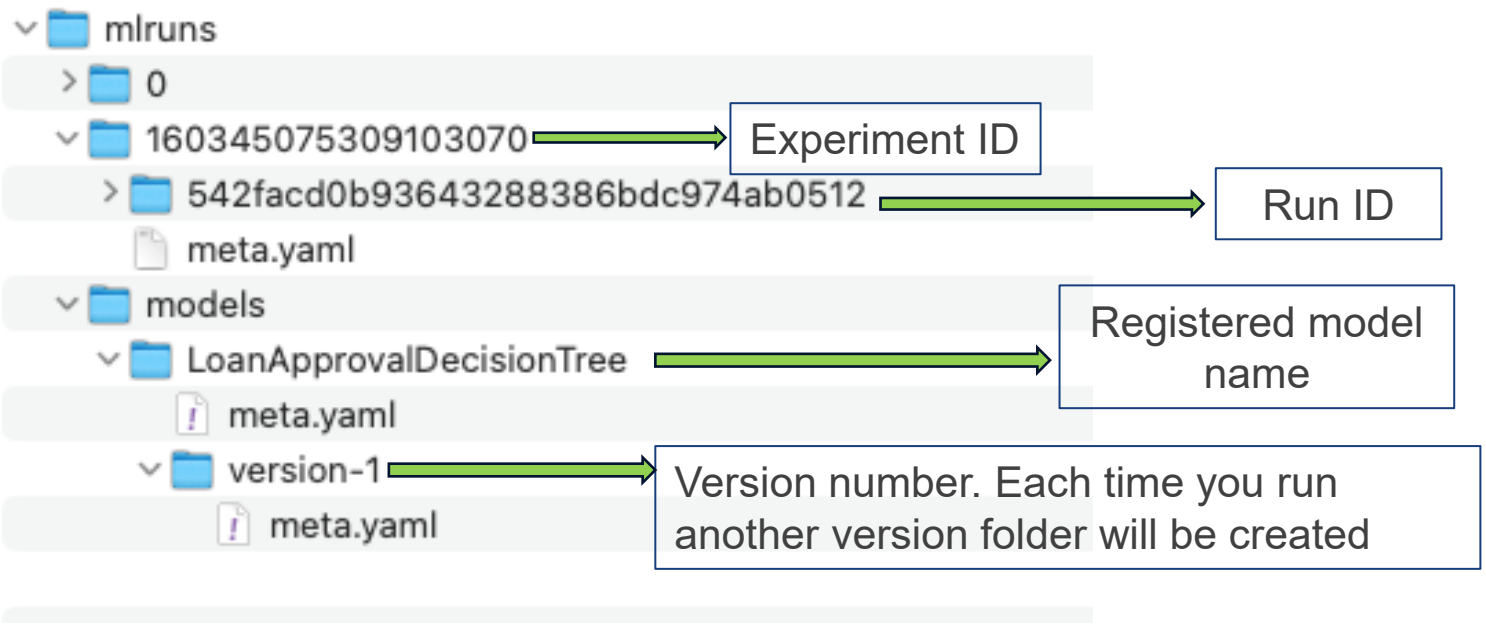
© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Register the model

```
# Make predictions on the test set
y_pred_test = dt_classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred_test)

# Logging hyperparameters, metrics
mlflow.log_param("max_depth", maxDepth)
mlflow.log_metric("accuracy_score", accuracy)

# Register the model
mlflow.sklearn.log_model(
dt_classifier,
artifact_path="loan_model",
registered_model_name="LoanApprovalDecisionTree"
)
```

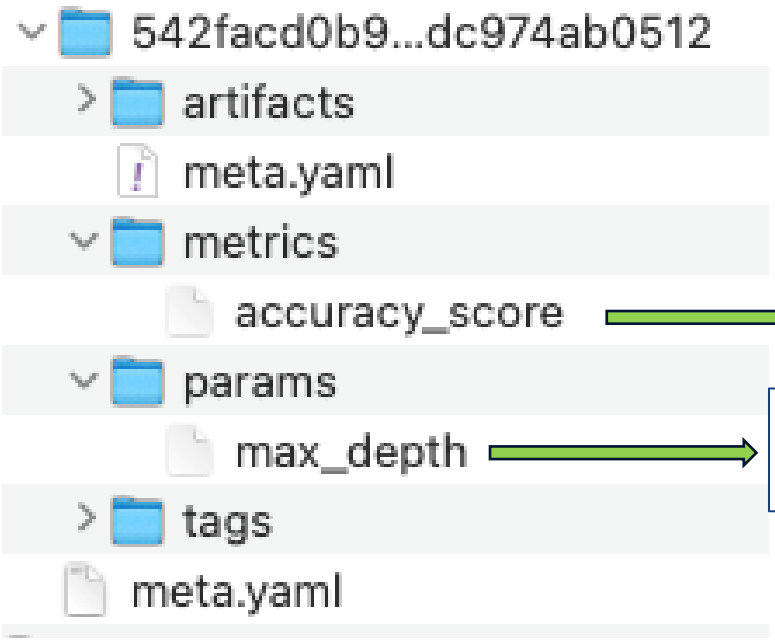
© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Explore the mlruns folder



© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Explore the Run Id folder



Storing the accuracy score of this experiment and run

Storing the hyperparameter of this experiment and run

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Load the registered model and make prediction

```
import mlflow.sklearn

model_name = "LoanApprovalDecisionTree"
model_version = "1"
# Load the model from the Model Registry
model_uri = f"models:/{model_name}/{model_version}"
model = mlflow.sklearn.load_model(model_uri)

# Generate a new dataset for prediction
X_new = pd.DataFrame({
'Age': [30, 45],
'Has_job': [1, 0],
'Own_house': [1, 0],
'Credit_rating': [2, 1]
})
predictions = model.predict(X_new)
print(predictions)
```

© 2011-24 National University of Singapore. All Rights Reserved.

Implement CI/CD Using MLFlow – Serve the model as RestfulAPI

Running the following command in terminal:

```
mlflow models serve -m "models:/LoanApprovalDecisionTree/8" -p 5000
```

Registered model name

Version number

Port number

If your environment doesn't have pyenv installed and using conda

Run the same command with extra option:

```
mlflow models serve -m "models:/LoanApprovalDecisionTree/8" -p 5000 --no-conda
```

Skip the conda environment

© 2011-24 National University of Singapore. All Rights Reserved.

Model Retraining Trigger

- **Scheduled Retraining**
  - Set to train on a specific schedule, such as nightly during low traffic hours with the current data
- **New Data Patterns and Trends**
  - New data patterns and trends in real-world settings that were not present in their original training datasets
- **Dips in Accuracy**
  - Data Drift - Input data distribution change
  - Target Drift - Label distribution changes over time
  - Concept Drift – Relationship between features and labels changes
- **Bugs and Unexpected Predictions**

© 2011-24 National University of Singapore. All Rights Reserved.

Security Best Practices

- **Authentication and Authorization**
  - Use Role-Based Access Control (RBAC) for all tools (e.g., MLFlow, Git, Airflow).
  - Integrate with SSO (e.g., Okta, Azure AD).
  - Restrict write or deploy permissions to trusted users and bots.
- **Secret Management**
  - Store secrets in Vault, AWS Secrets Manager, Azure Key Vault, or GCP Secret Manager.
  - Never hardcode API keys or credentials in source code or notebooks.
  - Use CI/CD environment variable injection (secrets.GITHUB_TOKEN, AIRFLOW__SECRETS_BACKEND).

© 2011-24 National University of Singapore. All Rights Reserved.

Security Best Practices

- **Static Application Security Testing (SAST)**
  - Uses tools like SonarQube or Fortify to scan for code smell and security vulnerabilities
- **Dynamic Application Security Testing (DAST)**
  - Uses tools like OWASP ZAP to identify vulnerabilities like injection, XSS, CSRF, insecure headers, etc., by simulating real attacks — without needing access to the source code.
- **Container Image Scan**
  - Scan the container's image and dependencies in it for vulnerabilities
  - Using open-source tool like Trivy

© 2011-24 National University of Singapore. All Rights Reserved.

Security Best Practices

- **Static Application Security Testing (SAST)**
  - Uses tools like SonarQube or Fortify to scan for code smell and security vulnerabilities
- **Dynamic Application Security Testing (DAST)**
  - Uses tools like OWASP ZAP to identify vulnerabilities like injection, XSS, CSRF, insecure headers, etc., by simulating real attacks — without needing access to the source code.
- **Container Image Scan**
  - Scan the container's image and dependencies in it for vulnerabilities
  - Using open-source tool like Trivy

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Strategies

- **Blue/Green**
  - Version 2 is live alongside Version 1 but switched off only after Version 2 is confirmed to be working right
- **Rolling update**
  - Version 2 would progressively replace Version 1 without downtime
- **Canary release**
  - Version 2 handles a small percentage of the overall request that comes to the service.
  - 95-5, 90-10 are popular
- **Shadowing**
  - Requests are sent both to Version 1 and Version 2 but Version 2 does not respond to the client

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- CI/CD in traditional software engineering
- CI/CD in MLOps
- Implement CI/CD using MLFlows
  - Experiment tracking
  - Save model into model registry
  - Load the model
  - Serve the model
- Model Retraining Trigger
- Best practices for securing the MLOps pipelines
- Deployment Strategies

© 2011-24 National University of Singapore. All Rights Reserved.

APPENDIX A – GITHUB ACTION

© 2011-25 National University of Singapore. All Rights Reserved.

GitHub Action

What is GitHub Action ?

- Automation framework built in GitHub
- Enable automation of development workflow such as CI/CD directly from repositories

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

Key Concepts

- Workflow
- Events
- Jobs
- Runners

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

Workflow

- A configurable automated process that runs one or more jobs
- Defined in YAML files under .github/workflows/.
- It can be triggered by an event or manually

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

Event

- A specific event that triggers or runs a workflow in your GitHub repository

- Some of the events are :
  - push → Run workflow on every commit push
  - pull_request → Run tests before merging code
  - release → triggers automated tasks like building and publishing artifacts, sending notifications, or updating external systems

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

Job

- GitHub Action jobs run in parallel by default.
- A GitHub Action workflow runs one or more jobs, each containing a set of steps that execute commands or actions.

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

Runner

- Servers that execute workflows when triggered
- Each runner can handle only one job at a time
- GitHub offers runners for Ubuntu Linux, Microsoft Windows, and MacOS to run your workflows

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

Developer Workflow Use Cases

- **Continuous Integration**
  - Run unit tests, linting, type-checking automatically on PRs
- **Continuous Deployment**
  - Deploy apps to AWS, Azure, GCP, or Kubernetes clusters
- **Automated Code Review**
  - Use bots to check coding style or security vulnerabilities
- **Scheduled Tasks**
  - Data backup, nightly builds, report generation
- **Mono repo Management**
  - Trigger workflows only for affected services (path filters)
- **Documentation Automation**
  - Auto-build and deploy documentation (e.g., mkdocs, Sphinx)

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD Pipeline with GitHub Actions

- **CI Stage (on pull_request or push)**
  - Checkout code (actions/checkout)
  - Install dependencies (pip install -r requirements.txt)
  - Run tests (pytest, jest, etc.)
  - Lint & static analysis (flake8, eslint)
- **CD Stage (on main branch merge or release)**
  - Build artifacts (Docker image, JAR, etc.)
  - Push artifacts to registry (GitHub Container Registry, ECR, GCP Artifact Registry)
  - Deploy to staging/production (via Terraform, Helm, or serverless deploy)

© 2011-24 National University of Singapore. All Rights Reserved.

CI/CD Pipeline with GitHub Actions

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt
      - run: pytest tests/

  deploy:
    needs: build-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - run: echo "Deploying to production..."
```

Workflow name

Event Type

Activity Type

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

MLOPS with GitHub Actions

- **Model Development & Testing**
  - Trigger training jobs on PRs or commits (using cloud runners or self-hosted GPU runners).
  - Run unit tests for ML pipelines (data validation, feature checks)
- **Model Training & Experiment Tracking**
  - Automate training pipelines (integrate with MLflow, Weights & Biases).
  - Store metrics and artifacts as workflow outputs.
- **Model Deployment**
  - Build & push model Docker images.
  - Deploy models to Kubernetes (Kubeflow, Seldon, BentoML).
  - Deploy serverless inference endpoints (AWS Lambda + Bedrock, Azure ML).
- **Model Monitoring**
  - Schedule workflows for drift detection, retraining, or evaluation on new data

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

MLOPS with GitHub Actions

- **Model Development & Testing**
  - Trigger training jobs on PRs or commits (using cloud runners or self-hosted GPU runners).
  - Run unit tests for ML pipelines (data validation, feature checks)
- **Model Training & Experiment Tracking**
  - Automate training pipelines (integrate with MLflow, Weights & Biases).
  - Store metrics and artifacts as workflow outputs.
- **Model Deployment**
  - Build & push model Docker images.
  - Deploy models to Kubernetes (Kubeflow, Seldon, BentoML).
  - Deploy serverless inference endpoints (AWS Lambda + Bedrock, Azure ML).
- **Model Monitoring**
  - Schedule workflows for drift detection, retraining, or evaluation on new data

© 2011-24 National University of Singapore. All Rights Reserved.

GitHub Action

MLOPS with GitHub Actions – Simple example

```yaml
name: MLOps Workflow

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.8'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: |
          pytest tests/
```

© 2011-24 National University of Singapore. All Rights Reserved.